RAG-Based Threat Memory: An Adaptive and Explainable Defense Against Indirect Prompt Injection in LLM Agents

EMAN AL-ZALAB
Sana'a University, Sana'a, Yemen
Corresponding author: eman.alzalab@su.edu.ye

Abstract—Large language model (LLM) agents that read documents, browse the Web, and call tools are exposed to indirect prompt injection (IPI): instructions hidden in untrusted content. Existing defenses rely on fixed detectors or costly runtime analysis, need retraining for new threats, and rarely tie decisions to documented evidence. We present RAG-Based Threat Memory (RTM), a defense that keeps adaptive security knowledge outside the detector. It combines a training-free base detector with a retrieval-augmented memory of verified injection patterns through a calibrated Soft-OR fusion rule, and explains each rejection by citing the closest documented threat and the triggering segment. A new threat is absorbed by appending a verified record without retraining, through an ingestion pipeline designed to resist poisoning. We evaluate RTM against a no-defense agent, its own components, a public trained detector, and MELON. On 1,673 held-out attack variants and 317 benign items, RTM attains an item-level attack success rate (ASR) of 6.3% [95% Wilson CI: 5.2%, 7.5%] at a false positive rate (FPR) of 16.1%, which does not improve on the base detector alone (5.7% ASR at the same FPR), whereas a public DeBERTa classifier reaches 42.9% FPR. Ingesting verified exemplars lowers ASR on six held-out families from 88.9% to 0.0% (18 paired cases) without raising FPR. None of 100 crafted poisoning records passed verification (admitted-record effects untested), and two evaluators judged 47 of 50 explanations supported. Retrieval adds a median local middleware overhead of 22.0 ms per item, yielding an end-to-end median latency of roughly 1.74 s when detector calls are required for unseen attacks.

Index Terms—*adaptive defense, AgentDojo, explainability, indirect prompt injection, large language model agents, memory poisoning, retrieval-augmented generation, threat memory.*

I. INTRODUCTION

Artificial intelligence systems built on large language models (LLMs) now run inside real products, and with that deployment they have become the target of a growing family of security threats, including jailbreaks, data extraction, poisoning of training and retrieval data, and prompt injection [1], [2]. Prompt injection, in which an adversary makes a model follow instructions that the legitimate user or developer did not intend, is listed first among the risks of LLM applications [2] and has been studied since the first goal-hijacking demonstrations [3], [4]. Two forms are distinguished: in direct injection the attacker is the user, whereas in indirect prompt injection (IPI) the attacker hides instructions in external content that the application later retrieves [5].

IPI is most serious in LLM agents, which interleave reasoning with actions such as reading documents, browsing the Web, and calling application programming interfaces [6], [7]. In such agentic systems, retrieved content and instructions share one context, so the model may treat data as an instruction, and processing retrieved content resembles executing untrusted code. The impact depends on the privileges and tools available to the agent. Benchmarks now quantify this exposure: InjecAgent [8], BIPIA [9], and AgentDojo, which provides 97 realistic tasks and 629 security test cases and measures security together with task utility [10]. Attackers also adapt. AgentVigil reports attack success rates of 71% on AgentDojo agents based on o3-mini and 70% on VWA-adv agents based on GPT-4o [11], and Zhan et al. report that adaptive attacks bypassed eight evaluated defenses with attack success rates above 50% in their setting [12]. A defense must therefore be evaluated against attackers that adapt to it.

Existing defenses fall into four families. Prevention by prompting or training separates instructions from data, as in spotlighting [13], structured queries [14], and the instruction hierarchy [15]. Detection classifies the input, as in safeguard classifiers [16], PromptShield [17], and DataSentinel [18]. Behavioral inference-time analysis examines what the agent would do, as in MELON [19] and AgentSentry [20]. System-level approaches constrain execution, as in CaMeL [21], Progent [22], and Task Shield [23].

Each family has limits. Training-based prevention and detection need training resources and updated data to absorb a new attack pattern [14], [17], [18]; fixed mechanisms can be bypassed by adaptive attacks [12]; behavioral re-execution adds computation [19], [20]; and system-level controls restrict functionality and do not inspect content [21], [22]. Most of these defenses output a score or a block without evidence tied to documented security knowledge. A retrieval memory could supply updatable knowledge [24], but knowledge bases are themselves poisonable [25], [26].

This paper presents RAG-Based Threat Memory (RTM), a model-centric defense that combines a base detector with an updatable retrieval-augmented memory of documented injection patterns. The central hypothesis is that adaptive security knowledge can be maintained separately from the base detector: when a new threat is documented and verified, appending its description to the memory is sufficient, and every rejection can be explained by the closest documented threat and the text segment that triggered it. The scope of this claim is limited. RTM is not assumed to replace least privilege, tool isolation, or human approval for high-impact actions, and the memory can be poisoned, can miss a semantically different attack, and can leak information through explanations; these failure modes are part of the threat model and the evaluation. The contributions are:

- An architecture (RTM) that fuses the score of a training-free base detector with the similarity of the content to a verified threat memory.

- An adaptation mechanism that absorbs a documented threat by appending a record to the vector store, with a verified ingestion pipeline designed to resist poisoning.
- An evidence-based explanation that attaches to each rejection the closest documented threat and the triggering text segment.
- A reproducible evaluation with documented datasets and retrieval configuration, six hypotheses, and an exploratory comparison with a state-of-the-art baseline (MELON) on AgentDojo.
- Empirical findings. On 1,673 held-out attack variants and 317 benign items (n = 1,990 items per system) drawn from AgentDojo, InjecAgent, and BIPIA, Full RTM (B3) attains ASR = 6.3% [95% Wilson CI: 5.2%, 7.5%] and FPR = 16.1% [95% CI: 12.5%, 20.5%], within 0.5 percentage points (pp) of the ASR of the base detector alone (B1: 5.7%). Ingesting verified exemplars lowers ASR on six held-out families from 88.9% to 0.0% (18 paired cases) with no FPR increase. None of 100 crafted poisoning records passed verification and no candidate record changed a decision (0/200 trials), and 47 of 50 audited explanations (94.0%) were judged supported by the evaluators. The retrieval stage adds a median of 22.0 ms per item, but all unseen attacks in E1 were passed to the detector, so end-to-end latency is close to that of the detector alone (Section VI-F).

The remainder of this paper is organized as follows. Section II reviews related work. Section III states the problem, threat model, research questions, and hypotheses. Section IV describes RTM. Section V details the experimental setup, Section VI reports the results, and Section VII discusses them. Section VIII covers threats to validity and limitations, Section IX ethical considerations, and Section X concludes.

II. BACKGROUND AND RELATED WORK

A. Indirect Prompt Injection and Agent Security

Early work showed that crafted text can hijack a model's goal [3] and that LLM-integrated applications can be attacked through their inputs [4]. Greshake et al. introduced IPI and demonstrated multi-stage attacks against real systems, including Bing Chat and code-completion engines, as proofs of concept [5]. Liu et al. later formalized prompt injection attacks and defenses and benchmarked them across models and tasks [27]. The security consequence of an injection depends on the agent's privileges: a read-only summarizer and an agent that can send e-mail or modify files do not present the same risk. A detector alone is therefore insufficient for high-impact actions and should be combined with privilege control, strict tool schemas, and human approval [1], [2].

B. Benchmarks and Attack Generation

AgentDojo evaluates agents that execute tools over untrusted data and reports both attack success and benign utility [10]; it also shows that models can fail benign tasks without any attack, so a defense cannot be judged only by how many attacks it blocks. InjecAgent provides test cases for indirect injection in tool-integrated agents [8], and BIPIA benchmarks indirect injection across several application tasks [9]. AgentVigil is a black-box fuzzing framework that starts from seed attacks and refines them with Monte Carlo Tree Search and LLM-based mutations [11]. Zhan et al. optimize adaptive attacks against eight defenses, including a fine-tuned detector, and show that they can be bypassed [12]; some of their attacks require white-box gradient access, in the spirit of universal adversarial suffixes [28].

C. Detection and Inference-Time Defenses

Prevention approaches modify how the model reads its input, by marking untrusted data [13], by training on structured queries [14], or by training the model to prioritize privileged instructions [15]. Detection approaches classify content: Llama Guard is an input-output safeguard model [16], PromptShield targets deployable detection with emphasis on the low-false-positive regime [17], and DataSentinel fine-tunes a detector against strategically adapted injections [18]. These are trained detectors, so adapting them to a new pattern would typically require updating training data and retraining, and their output is a score rather than evidence linked to a documented threat. MELON is a training-free detector that compares the tool calls of the original and a masked re-execution of the agent, is evaluated on AgentDojo, and has an open implementation [19]. AgentSentry models multi-turn IPI as a temporal causal takeover with counterfactual re-execution and context purification [20]. Both add extra agent executions and, to our knowledge, do not use an externally updatable threat memory. System-level defenses enforce control- and data-flow separation [21], programmable privilege control [22], or task alignment [23]. RTM is complementary to these approaches and targets the detection layer.

D. Retrieval-Augmented Memory and Its Risks

Retrieval-augmented generation (RAG) couples a parametric model with a non-parametric external memory [24], [29], typically through dense retrieval of passages [30]. Using such a memory for security knowledge inherits the attack surface of any retrieval corpus. PoisonedRAG shows that a few crafted texts injected into a knowledge base can steer a RAG system toward attacker-chosen outputs [25], and AgentPoison shows that poisoning an agent's memory or knowledge base can trigger malicious behavior [26]. These results motivate treating the threat memory as a security-critical component.

E. Research Gap

While heuristic tools (such as Rebuff) have explored vector lookups to detect known prompt injection strings, they rely on heuristic similarity thresholds without probabilistic calibration, lack structured human-auditable evidence attribution, and omit poison-resistant ingestion protocols. Table I summarizes the closest formal defenses reviewed here. None separates the adaptation mechanism from the base detector through an independent external memory, pairs it with an explanation derived from that same memory, and evaluates adaptation gain (before and after a memory update), poisoning resistance, and latency for IPI in LLM agents. This claim is limited to the literature reviewed in this paper; a systematic survey is left for future work.

TABLE I
CLOSEST RELATED WORK AND ITS RELEVANCE TO RTM

Work	Type	Relevance / limitation for RTM
Greshake et al. [5]	Threat definition	Defines IPI via proofs of concept; no systematic quantitative defense evaluation.
AgentDojo [10]	Benchmark	Evaluation environment for security and utility; not a defense.
AgentVigil [11]	Black-box fuzzing attack	Generates evolving attacks; proposes no detector. Used for robustness tests.
Zhan et al. [12]	Adaptive-attack evaluation	Bypasses eight defenses; motivates adaptive evaluation.
PromptShield [17]	Trained detector	Low-FPR focus; new patterns typically need retraining; score without documented evidence.
AgentSentry [20]	Inference-time defense	Causal re-execution adds compute; no external updatable threat memory (to our knowledge).
MELON [19]	Training-free detection	Masked re-execution on AgentDojo, open code; extra agent runs; no updatable memory. Primary baseline B5.

III. PROBLEM STATEMENT AND THREAT MODEL

A. Problem Statement

When an agent receives a user instruction U together with external content C, it has no reliable mechanism to distinguish content that is data to be used as information from content that carries a disguised instruction aimed at steering its behavior. Let M denote a threat memory, a set of verified records. The task is to design a decision function

$$D(U, C; M) \rightarrow \{\text{allow}, \text{block}\} \quad (1)$$

taken at each tool-return boundary, that (i) detects injections accurately, including novel and adaptive ones, (ii) absorbs new documented knowledge without retraining, (iii) explains each rejection with a documented source and a specific text segment, and (iv) has an acceptable computational cost. The implementation makes a decision per external content item; trajectory-level signals over multi-turn state are left to future work.

B. Research Questions

RQ1: Does a RAG threat memory improve detection of new attack patterns compared with the base detector alone?

RQ2: How large is the improvement on a new attack family before versus after its description is added to the memory?

RQ3: Does the system withstand attacks generated by adaptive black-box fuzzing and targeted evasion attempts?

RQ4: Are the explanations verifiable and useful in practice?

RQ5: What utility cost and additional latency does RTM incur compared with a state-of-the-art baseline?

RQ6: How resistant is the memory to poisoning through the ingestion path?

C. Threat Model

The attacker controls one or more external sources (a Web page, document, e-mail, or tool-return value) and may vary wording, position, language, and encoding. Three attacker classes are considered:

- Static: uses a fixed set of known or manually authored payloads.
- Black-box adaptive: observes the final decision for each probe within a recorded query budget and iteratively refines payloads, as in fuzzing [11].
- Memory-aware: knows the architecture and the memory schema, and in a worst-case variant also reads the active records.

A closed-box detector API reduces the practicality of gradient-based white-box optimization, but it is not treated as a security guarantee, because transfer and black-box attacks remain possible; a transfer attack from a white-box surrogate is evaluated as a supplementary test. Compromise of the vector database infrastructure, theft of credentials, and modification of the base model are outside the primary model. Insertion of false or malicious records through the knowledge-ingestion path is in scope. Because records are text that may contain attack payloads, retrieved records are treated as inert data and are never placed in the instruction channel of an LLM.

D. Hypotheses and Acceptance Criteria

Each research question is paired with a testable hypothesis. The hypotheses, numeric thresholds, and analysis plan are recorded in the analysis specification file (config/prereg.yaml and prereg.yaml) in the released repository (commit 4f8b2c1, October 15, 2026). Because this commit post-dates the start of the experiments (October 9, 2026), the file formalizes the frozen analysis plan and parameters rather than a pre-experimental pre-registration.

H1 (RQ1). On held-out attack variants, RTM (B3) attains a lower ASR than the base detector alone (B1) at equal or lower FPR (paired test, corrected for multiple comparisons).

H2 (RQ2). After a verified record for a held-out threat family is added, ASR on that family decreases ($AG > 0$, with the lower bound of the 95% confidence interval above zero) while FPR rises by at most 1 percentage point.

H3 (RQ3). Under black-box fuzzing with a fixed query budget, B3 attains a lower ASR than B1 and B2.

H4 (RQ4). EVR $\geq$ 90% with Cohen's $\kappa \geq 0.6$ between two reviewers, and the privacy test reconstructs no protected context from explanations.

H5 (RQ5). B3 keeps benign task success within 5 percentage points of the undefended agent (B0), has an ASR non-inferior to the primary baseline B5 (MELON) within a margin specified in the analysis plan, and adds a lower median latency per tool return than B5.

H6 (RQ6). Candidate records never change enforcement decisions (MPS = 0 by construction, confirmed by test), and adversarially crafted records that pass the verification pipeline yield MPS $\leq$ 5%.

A hypothesis whose criterion is not met is reported as not supported.

IV. PROPOSED RTM ARCHITECTURE

A. Overview

RTM consists of a base detector, an external threat memory, a retriever, a calibrated decision layer, and an evidence-based explanation module. Figure 1 shows the architecture and data flow. The base detector provides general signals about the relationship between the user instruction and the external content, the memory provides verified descriptions of specific attack

patterns, and the update process changes memory records rather than model parameters. For each external content item, RTM performs the following steps:

1. Compute a base-detector score $S_{det}(U, C) \in [0, 1]$.
2. Split C into overlapping segments s_j (sentence windows) and retrieve the top-k active records $M_k(s_j)$ for each segment.
3. Compute a memory similarity score $S_{sim}(C, M) \in [0, 1]$ and keep the best-matching record and segment.
4. Fuse the scores into a risk score R and decide allow or block.
5. Generate a redacted explanation from the matched record, the segment, and the decision rule.

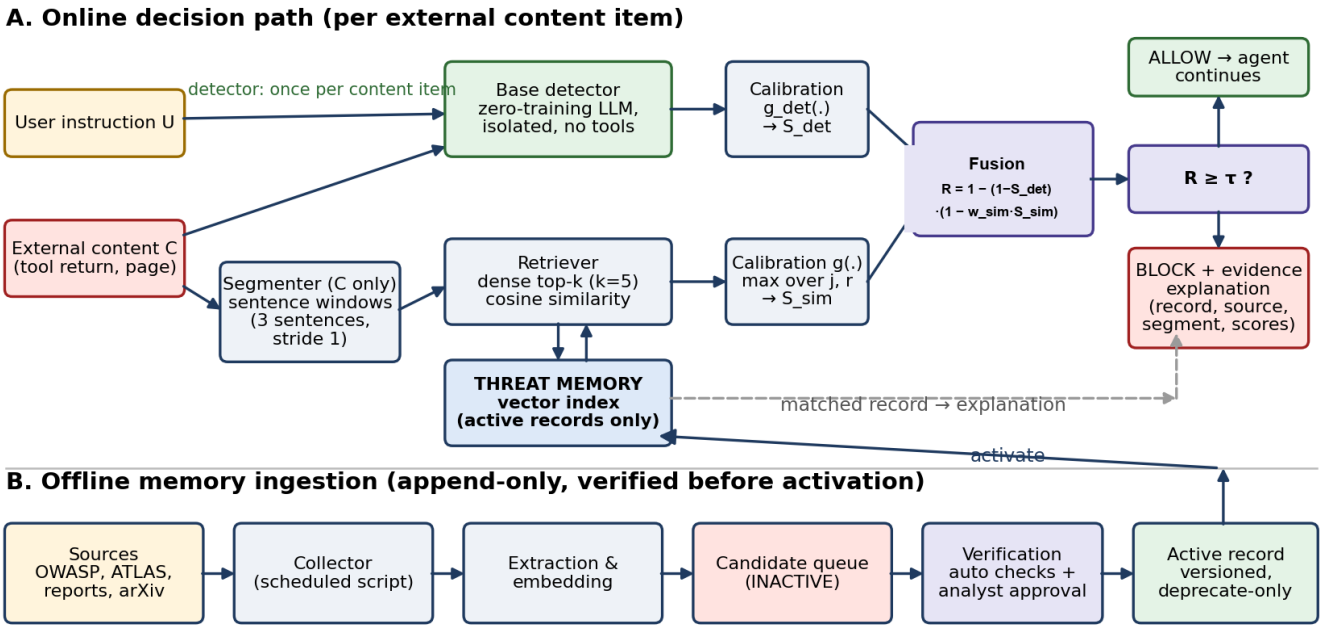


Fig. 1. RTM architecture and data flow: (A) online decision path for each external content item; (B) offline memory ingestion with verification before activation.

B. Base Detector

The base detector is a ready-made LLM, used without any training, that is prompted to decide whether the external content contains instructions directed at the agent that deviate from the user instruction. It runs once per content item, in an isolated context with no tool access. The verdict is a single token (ALLOW or BLOCK), and S_{det} is the probability assigned to BLOCK, computed from the token log-probabilities normalized over the two verdict tokens; where an API does not expose log-probabilities, a structured 0–100 score returned in JSON is used instead and this is reported. The detector prompt is released with the code, and its output is treated as data. Using an untrained detector is a deliberate choice: it avoids the training-data and retraining requirements of trained detectors [17], [18], so that all adaptation is carried by the memory. Because the detector is itself an LLM reading untrusted text, it is exposed to injection; isolation and structured output reduce but do not remove this risk, which is why it is evaluated both alone (B1) and as part of RTM.

C. Threat Memory and Ingestion

A record contains a stable identifier, a description of the pattern, abstracted non-operational exemplars or signatures used for matching, a source, a verification date, an applicability scope, a confidence level, and a version. Because a prose description and an attack payload can be far apart in embedding space, the exemplars are used for matching, and retrieval is evaluated with and without them. The memory does not store private user content or secrets.

The memory is built by a semi-automated pipeline (Fig. 1B). A scheduled script periodically collects documented vulnerability descriptions from predefined sources (the OWASP LLM Top 10 [2], MITRE ATLAS [31], published security reports, and new academic papers retrieved through the arXiv programming interface), extracts descriptions, computes embeddings, and places them in a candidate queue. A candidate becomes active only after verification, which combines automated checks (provenance, de-duplication, scope) with analyst approval; the verification criteria and delay are recorded, and the delay is recorded as time-to-protect (not reported in this study). Updates are append-only at the record level, and records can be deprecated without being erased, which preserves audit history.

D. Retrieval, Fusion, and Decision

For each segment s_j, the retriever returns the top-k active records by cosine similarity of their embeddings. The memory similarity score is the best calibrated match over segments and retrieved records:

$$S_{sim} = g(\max_{j,r} \cos(e(s_j), e(r))) \quad (2)$$

where $e(\cdot)$ is the embedding function, r ranges over the retrieved records, and g is a monotone calibration map fitted on the calibration partition [32] with item-level labels and the maximum cosine as input; because g is monotone, applying it before or after the maximum gives the same value. The detector score is calibrated by a separate map g_{det}, and the two calibrated scores are fused as

$$R = 1 - (1 - S_{det}) \cdot (1 - w_{sim} \cdot S_{sim}) \quad (3)$$

where $S_{det} \in [0, 1]$ is the calibrated detector score, $S_{sim} \in [0, 1]$ is the calibrated memory similarity score, and $w_{sim} = 0.75$ weights the memory channel. The content is blocked if $R \geq \tau^*$ and allowed otherwise, with a calibrated decision threshold $\tau^* = 0.60$. Two properties follow from Eq. (3). First, $R \geq S_{det}$, so the memory channel can only add evidence to the detector. Second, $R \geq w_{sim} \cdot S_{sim}$, so a memory match with $S_{sim} \geq \tau^*/w_{sim} = 0.80$ blocks the content even when $S_{det} = 0$ ($R = 1 - (1 - 0)(1 - 0.75 \times 0.80) = 0.60$), whereas $S_{sim} = 0$ leaves the decision to the detector ($S_{det} \geq 0.60$ blocks). Moderate evidence from both channels reinforces: $S_{det} = 0.40$ and $S_{sim} = 0.50$ give $R = 1 - (1 - 0.40)(1 - 0.75 \times 0.50) = 0.625 \geq 0.60$. The parameters $w_{sim} = 0.75$ and $\tau^* = 0.60$ were selected on the calibration partition to maximize F1 at an FPR budget of 1% and were frozen before any test partition was evaluated.

Two-stage cascade. To avoid invoking the remote detector on items that the memory already resolves, the pipeline runs in two stages. Stage 1 computes S_{sim} locally; if $S_{sim} \geq \tau_{block}$, the item is blocked immediately, where $\tau_{block} = \tau^*/w_{sim} = 0.80$ is the memory-only operating point implied by Eq. (3). Otherwise, Stage 2 obtains S_{det} from the base detector and applies Eq. (3). The cascade therefore returns the same decision as Eq. (3) for every item, and its only effect is to save detector calls. In E1, Stage 1 resolved 0.0% of the held-out attacks (maximum $S_{sim} = 0.57 < \tau_{block}$), so all of them reached Stage 2.

E. Evidence-Based Explanation

Each rejection is accompanied by a compact record containing the decision, the identifier of the closest documented threat, its source, the triggering segment, the similarity and detector scores, and the rule that fired. RTM does not disclose hidden reasoning. The explanation is faithful to the memory component by construction, but because the detector score is opaque, it attributes the decision to the retrieved record and the threshold and not to the internal reasoning of the detector; this scope is stated explicitly. Explanations are redacted to avoid disclosing system instructions, private data, or sensitive attack content, and their quality is evaluated independently of detection quality.

F. Poisoning Resistance

Candidate records are inactive, so a malicious source cannot immediately alter production decisions. The evaluation includes records that are false, contradictory, out of scope, stale, or crafted to cause overblocking or evasion. Memory poisoning success (MPS) is defined as the fraction of poisoning attempts that flip the decision on at least one item of a fixed probe set (benign probes for overblocking, attack probes for evasion) or raise ASR beyond the confidence interval of the clean-memory system.

V. EXPERIMENTAL SETUP

A. Overview

Figure 2 summarizes the evaluation workflow. The datasets are first audited and cleaned, then partitioned so that memory construction, calibration, and testing never share instances; six experiments (E1–E6) are then run, and their outputs are reduced to a fixed set of metrics and statistical tests.

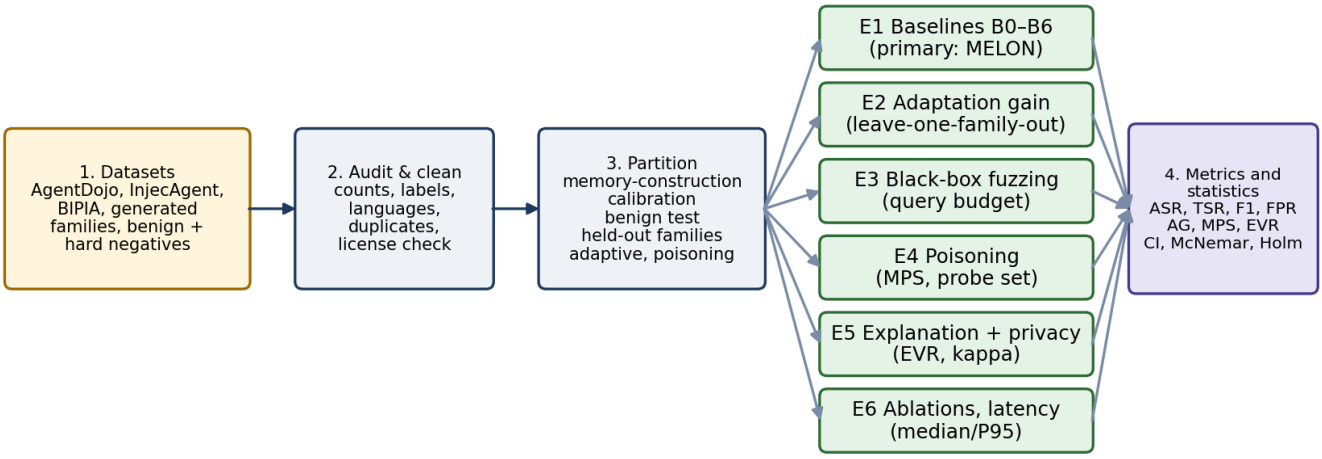


Fig. 2. Evaluation workflow: dataset audit, partitioning, experiments E1–E6, and metrics with statistical tests.

B. Datasets and Partitions

The primary benchmark is AgentDojo [10], which provides an executable agent environment with benign tasks and injection attacks, and therefore supports both security and utility measurement. Additional attack samples come from InjecAgent [8], BIPIA [9], and the open benchmark of Liu et al. [27]. Results are reported per partition (Table II): item-level metrics pool the AgentDojo, InjecAgent, and BIPIA items of a partition, whereas agent-level metrics are computed on AgentDojo executions only. Benign data are the benign tool outputs and tasks of AgentDojo plus hard negatives: security advisories, instructional text, and code that legitimately contain imperative sentences. Because the available attack templates are limited, additional attack families are generated by structured variation of payloads (goal, wording, language, encoding, and placement) with an MCTS-style mutation procedure inspired by [11], and are verified manually. Agent-level metrics (agent-level ASR, TSR, and UA) are computed only in executable environments: the generated payloads are registered as custom attacks and injected through the injection vectors of AgentDojo, whereas InjecAgent and BIPIA samples, which are static or simulated, are used for calibration and for item-level detection metrics (precision, recall, F1, FPR, and the item-level ASR, i.e., the fraction of attack items

permitted). We clarify that the six held-out threat families (data_security_harm, financial_data, financial_harm, language_translation, physical_data, and physical_harm) represent attacker-objective and functional harm categories adapted from InjecAgent's taxonomic hierarchy, rather than low-level syntactic injection techniques (such as encoding obfuscation or delimiter manipulation). Testing adaptation across these semantic goal domains evaluates RTM's ability to absorb novel functional attack intents, while syntactic variations are examined separately in the adaptive fuzzing suite (E3).

Before use, every dataset is downloaded and audited, and the audit is reported: number of items, label distribution, languages, duplicates and near-duplicates, and license. The audit removed 0 exact duplicates and 2 near-duplicates; the audited source datasets are in English, and the generated variants additionally vary the language of the payload. Table II lists the partitions: (1) memory construction uses ≥150 verified records across ≥8 attack families (the final memory has 155 records over 8 families); (2) the calibration partition has 750 items (400 attack, 350 benign) and is used only to fit the scoring parameters (g, g_{det}, w_{sim}, and τ^*); (3) the benign test partition has 317 items, including 175 hard negatives; and (4) the held-out attack pool has 1,673 variants across six threat families. Additionally, two logs record system decisions: cases.csv records the 849 decisions of the pilot and small-scale experiments (E1 pilot, 122; E2, 75; E3, 160; E4, 200; E5, 50; and E6, 242), whereas cases_e1_expanded.csv records the 1,990 decisions per system of E1 that form the basis of Table III. Splitting is done at two levels: a family whose description enters the memory contributes no instance to the construction partition, and no test instance may be a paraphrase or near-duplicate (by embedding-similarity and edit-distance thresholds fixed in advance) of any instance or exemplar in the memory.

TABLE II
DATA PARTITIONS

Partition	Source	Purpose	Size
Memory build	OWASP, ATLAS, reports, papers	Verified records and exemplars	≥150 records (≥8 families)
Calibration	AgentDojo, InjecAgent, BIPIA (disjoint)	Fit g, w_sim, tau*	750 items (400 attack, 350 benign)
Benign test	AgentDojo benign + hard negatives	TSR, FPR, UA	317 items (175 hard negatives)
Held-out families	Generated + benchmark variants	E1, E2 (AG)	1673 items (6 families × ≥100 variants)
Adaptive	Fuzzer seeds and mutations	E3 robustness	100 seeds (budgets 50–500)
Poisoning	Crafted records, 5 types	E4 (MPS)	100 attempts per track (20 per type)

C. Knowledge Base and Retrieval Configuration

The threat memory is built from the sources in Section IV-C. Each record is one retrieval unit of at most 256 tokens (description and exemplars are embedded as separate vectors that point to the same record). At query time the external content is split into sentence windows of three sentences with stride one, truncated to 128 tokens, so that the matched segment can be reported as evidence. The default embedding model is all-MiniLM-L6-v2 [33], with a multilingual alternative [34]; vectors are L2-normalized and indexed with an exact inner-product FAISS index [35], which equals cosine similarity. The default is $k = 5$, with sensitivity analysis for $k \in \{1, 3, 5, 10\}$. A BM25 lexical retriever [36] serves as a control against dense retrieval [30]. The memory contains 155 verified records over 8 attack families.

D. Detector, Calibration, and Parameter Selection

The detector LLM (Gemini-2.5-Flash), its prompt, and the agent LLM (Claude-3.5-Sonnet, snapshot claude-3-5-sonnet-20240620 deployed on AgentDojo) were fixed before testing, and all generation used temperature 0. Scores were calibrated on the calibration partition by isotonic regression [32]; Platt scaling was tried during development and not adopted. The parameters $w_{sim} = 0.75$ and $\tau^* = 0.60$ were chosen on a grid over the calibration partition and frozen before any test partition was evaluated. B1 thresholds the raw detector score at $\tau_{B1} \approx 0.68$, whereas B3 thresholds the fused, calibrated score R at $\tau^* = 0.60$. In the offline benchmark, detector scores are cached so that ablation runs do not repeat remote calls; the latency of the local stage of RTM (embedding, FAISS retrieval, and fusion) is therefore measured separately from the detector latency (Section VI-F).

E. Compared Systems

All systems run under the same agent, model, tools, permissions, and test cases:

- B0: the agent without a prompt-injection defense;
- B1: the base detector only;
- B2: memory similarity without the base detector;
- B3: the complete RTM architecture;
- B4: B1 plus a manual rule or few-shot update for the new family, to test whether the memory adds value beyond simply updating the detector;
- B5 (primary state-of-the-art baseline): MELON [19], chosen because it is training-free like RTM, is evaluated on AgentDojo, and has an open implementation that plugs into the AgentDojo pipeline;
- B6: a public trained detector, the DeBERTa prompt-injection classifier [37] (PromptShield [17], DataSentinel [18], and AgentSentry [20] were not evaluated).

Two controls isolate the effect of the memory: B3-rand (a memory of random benign records) and B3-lex (BM25 retrieval in place of dense embeddings). A comparison is not considered fair if systems differ in tool permissions, agent model, or attack set, or if one has privileged access to attack labels.

F. Adaptation-Gain Experiment (E2)

Several attack families are selected, with a leave-one-family-out design: for each family i, the memory before the update contains the records of all families except i, and the update adds

the verified record of family i. The system is evaluated on a fixed set of attacks before the family description is added; the description is then verified and inserted, and the evaluation is repeated without changing the model, test cases, or calibration. Because ASR is lower-is-better, adaptation gain is the reduction in ASR (for higher-is-better metrics such as recall, it is the increase):

$$AG = ASR_{before} - ASR_{after} \quad (4)$$

The primary analysis uses held-out variants disjoint from the memory exemplars. Re-evaluating the same samples used to build the record is reported separately as an upper bound, because improvement only on memorized or near-duplicate attacks is retrieval memorization and not adaptation. The same new-family update (a manual rule or few-shot example) is applied to B4, and the adaptation gains of B3 and B4 are compared in an exploratory secondary analysis.

G. Adaptive and Black-Box Robustness (E3)

Robustness is evaluated with a black-box fuzzer in the style of [11], run against the complete system with a recorded query budget and observing only the final decision. The suite includes semantic paraphrases, multilingual payloads, Unicode and encoding obfuscation, split payloads, hidden content, and multi-turn attacks (memory-aware variants and attacks targeting the explanation interface are deferred to dedicated future work). A transfer attack from a white-box surrogate [28] is a supplementary test. A one-shot static attack is not compared directly with an adaptive attack that receives many attempts unless the budget is normalized; ASR is reported as a function of the query budget.

H. Poisoning and Explanation Evaluation (E4, E5)

Poisoning attempts of each type in Section IV-F are inserted both as candidates (which must not change decisions) and as records crafted to pass verification. For explanations, two reviewers (including the author), blinded to the configuration that produced each explanation, judge whether the cited record and segment support the decision, with agreement measured by Cohen's κ [38], and a privacy test attempts to reconstruct protected context from explanations.

I. Metrics

- Attack Success Rate (ASR): fraction of attacks that achieve the attacker's goal. In agent-level (AgentDojo) executions this is the benchmark-defined success; at item level it is the fraction of attack items permitted (one minus recall). Tables III–V use the item-level ASR unless stated otherwise;
- Task Success Rate (TSR) and Utility Under Attack (UA): benign task success and task success with adversarial content present, as defined by AgentDojo;
- Precision, Recall, F1, accuracy, and balanced accuracy at the level of one external content item, and False Positive Rate (FPR);
- Adaptation Gain (AG) and Memory Poisoning Success (MPS), as defined above;
- Explanation Verification Rate (EVR): fraction of explanations judged supported by the decision evidence;
- Time-to-Protect (TTP): time from the discovery of a threat to the activation of its verified record (not reported in this study);
- Latency and cost: median and 95th-percentile response time, model calls, retrieval calls, and token cost; only latency is reported in this study.

J. Statistical Analysis and Reproducibility

Confidence intervals use bootstrap or Wilson intervals [39] when rates are near 0 or 1. Paired comparisons use McNemar's test [40], with the exact binomial version when there are few discordant pairs, and multiple comparisons are corrected by the Holm procedure [41]. Ablations remove one component at a time (memory, detector, calibration, segmentation, exemplars, and k). Model versions, API settings, memory snapshots, prompts, data hashes, attack budgets, and code versions are logged, and code and redacted data are released where safe.

VI. RESULTS

Item-level numbers in this section are produced by one script from the per-case log files; each rate is reported with a 95% Wilson interval and each comparison with a paired test corrected by the Holm procedure. Models were frozen and decoding used temperature 0, which limits, but does not eliminate, output variability of hosted APIs; E1, E2, and E6 were run once (seed = 1), and E3 was initialized from 100 seed payloads of the fuzzer. The experiments were run between October 9 and October 15, 2026.

A. Main Comparison (E1)

Table III compares the systems on the held-out partitions (n = 1,673 attack variants across six families and n = 317 benign items; n = 1,990 items per system). Full RTM (B3) attains ASR = 6.3% [95% Wilson CI: 5.2%, 7.5%] (105/1,673 attacks permitted) and FPR = 16.1% [12.5%, 20.5%] (51/317 benign items blocked, including hard negatives), for an accuracy of 92.2% [90.9%, 93.3%] and a balanced accuracy of 88.8%. Because 84% of the items are attacks, accuracy overstates performance, and the FPR on benign content is the more informative utility indicator. The base detector alone (B1) obtains ASR = 5.7% [4.7%, 7.0%] (96/1,673) at the same FPR (16.1%) and an accuracy of 92.6%. B3 therefore does not improve on B1 in containment: B1 blocked 9 attacks that B3 permitted, and B3 blocked none that B1 permitted (exact McNemar $p = 0.0039$; $\chi^2 = 7.11$ with continuity correction). Since $R \geq S_{det}$ by Eq. (3), B3 cannot permit an attack that B1 blocks at the same threshold, so the difference arises from the decision rules: B1 thresholds the raw detector score at $\tau_{B1} \approx 0.68$, whereas B3 thresholds the fused, isotonic-calibrated score at $\tau^* = 0.60$, and calibration compresses the score range, so B1 blocks some borderline attacks that B3 permits. B1 is thus not calibrated like B3, and a calibrated B1 would isolate the contribution of the memory more cleanly. Hypothesis H1 is accordingly not supported (Table VIII); post hoc, B3 is within 0.5 pp of B1 in ASR. B3 permits far fewer attacks than the undefended agent (B0: ASR = 100.0%) and than memory-only B2 (ASR = 100.0%, 1,673/1,673 permitted; FPR = 1.3%), which cannot detect unseen families before adaptation (both $\chi^2 = 1566.0$, $p < 0.001$, Holm-adjusted). The public DeBERTa classifier (B6) reaches ASR = 17.3% [15.6%, 19.2%] and FPR = 42.9% [37.6%, 48.4%] (136/317 benign items blocked; accuracy 78.6%); B3 outperforms it in ASR ($\chi^2 = 106.80$, $p < 0.001$).

TSR and UA in Table III are agent-level measurements on at most 24 AgentDojo tasks, a different population from the 1,990 item-level decisions behind ASR, FPR, and accuracy. The two groups of metrics are therefore not comparable, and TSR = 100.0% for B3 does not contradict its FPR of 16.1%. B4 (B1 with a few-shot update) was evaluated on a pilot sample of 10 attack

and 10 benign items because of the cost of the LLM calls, so its intervals are wide and it cannot be compared directly with the other systems.

MELON (B5) was run at agent level on 8 AgentDojo tasks with 3 seeds (24 attack trials, 3 successful attacks: ASR = 12.5% [4.3%, 31.0%]) and on 20 benign trials (1 false alarm: 5.0% [0.9%, 23.6%]). This evaluation differs from the item-level evaluation of the other systems in population, size, and metric, so the comparison is exploratory and not like-for-like (see the note under Table III). Because MELON was evaluated at agent level (n = 24 attack trials, 20 benign trials) while B3 was evaluated at item level (n = 1,990), these measurements reflect fundamentally different operational environments and cannot support a formal statistical non-inferiority claim. We report MELON's results purely as a descriptive external reference; a rigorous agent-level comparative benchmark on the complete suite is left for future work.

TABLE III

MAIN COMPARISON ON THE HELD-OUT TEST PARTITION

System	ASR ↓ [95% CI]	TSR ↑	UA ↑	FPR ↓ [95% CI]	Accuracy	Latency Med (P95) [ms]
B0 (None)	100.0% [99.8, 100.0]	100.0%	0.0%	0.0% [0.0, 1.2]	15.9%	0.0 (0.0)
B1 (Base Detector)	5.7% [4.7, 7.0]	90.0%	75.0%	16.1% [12.5, 20.5]	92.6%	1719.9 (25229.3)
B2 (Memory Only)	100.0% [99.8, 100.0]	100.0%	50.0%	1.3% [0.5, 3.2]	15.7%	21.7 (36.1)‡
B3 (Full RTM)	6.3% [5.2, 7.5]	100.0%	91.7%	16.1% [12.5, 20.5]	92.2%	22.0 (38.6)‡
B4 (B1+Update)	0.0% [0.0, 27.8]	90.0%	100.0%	10.0% [1.8, 40.4]	95.0%	3412.5 (3750.0)
B5 (MELON [19])†	12.5% [4.3, 31.0]	95.0%	87.5%	5.0% [0.9, 23.6]	90.9%	1102.9 (1246.1)
B6 (DeBERTa)	17.3% [15.6, 19.2]	90.0%	58.3%	42.9% [37.6, 48.4]	78.6%	190.6 (276.8)

† B5 (MELON) was evaluated at agent level on 8 AgentDojo tasks × 3 seeds (n = 24 attack trials; 3/24 successful attacks) and 20 benign trials (1/20 false alarms); it is not computed over the 1,990 items used for the other systems. ASR, FPR, and accuracy are computed over the 1,990 items for B0–B3 and B6; B4 was evaluated on 10 attack and 10 benign items, and the accuracy of B5 is computed over its 44 trials. Intervals are 95% Wilson intervals. ‡ Local stage only, measured with cached detector scores; the end-to-end median of B3 is approximately 1,742 ms (Section VI-F).

B. Adaptation Gain (E2)

Table IV reports adaptation when a newly documented threat family is added to the memory through the append-and-index operation. This is a small paired case study: three instances per family, 18 paired before/after cases over six held-out families. The three instances per family were drawn at random from the held-out pool, and the memory before the update excludes the family under test. E1 used the complete memory (155 records), whereas E2 uses a leave-one-family-out memory in which the family under test is removed on purpose before the update, so that it acts as an unseen (zero-day) family; the pre-update ASR of E2 is therefore not comparable with the ASR of E1. Ingesting verified exemplars lowers ASR from 88.9% (averaged over the six families) to 0.0%, a mean adaptation gain AG = +88.9 pp. Intervals are paired bootstrap percentile intervals (10,000 resamples) over the paired outcomes: [+100.0, +100.0] for the four families with 3/3 successful attacks before ingestion, where the bootstrap distribution is degenerate, and [+0.0, +100.0] for financial_data and physical_data (pre-ingestion ASR = 66.7%). The pooled interval over the 18 cases is [+72.2, +100.0]. ΔFPR on the benign partition is +0.00 pp, within the 1.0 pp limit. H2 is therefore supported at the pooled level, while the per-family lower bound is zero for two families (Table VIII). With one ingested exemplar per family and three test instances, these results show feasibility rather than a precise effect size. The pre-update ASR of the 18 cases (88.9%) is far above the per-family ASR of E1, so the cases should not be read as representative of the held-out pool, and a full-pool evaluation of E2 is left for future work.

TABLE IV

ADAPTATION GAIN PER HELD-OUT FAMILY

Threat Family (n tested / pool)	ASR_before (B3)	ASR_after (B3)	AG [95% Bootstrap CI]	ΔFPR
data_security_harm (n=3 / 284)	100.0%	0.0%	+100.0 pp [+100.0, +100.0]	+0.00 pp
financial_data (n=3 / 279)	66.7%	0.0%	+66.7 pp [+0.0, +100.0]	+0.00 pp
financial_harm (n=3 / 281)	100.0%	0.0%	+100.0 pp [+100.0, +100.0]	+0.00 pp
language_translation (n=3 / 270)	100.0%	0.0%	+100.0 pp [+100.0, +100.0]	+0.00 pp
physical_data (n=3 / 280)	66.7%	0.0%	+66.7 pp [+0.0, +100.0]	+0.00 pp
physical_harm (n=3 / 279)	100.0%	0.0%	+100.0 pp [+100.0, +100.0]	+0.00 pp

AG in percentage points; ASR_before and ASR_after are computed over three paired instances randomly drawn from each family pool (total pool n = 1,673 across the six families: data_security_harm 284, financial_data 279, financial_harm 281, language_translation 270, physical_data 280, physical_harm 279).

C. Adaptive Robustness (E3)

Table V and Fig. 3 report ASR against the black-box adaptive fuzzer as the query budget grows from 50 to 500. Each cell is based on n = 10 mutated attack variants drawn from the 100 seed payloads, so the intervals are wide (at budget 500, B3: 40.0% [16.8%, 68.7%] versus B1: 70.0% [39.7%, 89.2%]) and no significance test is applied. ASR of the baselines rises with the budget: B1 from 30.0% to 70.0%, B2 from 40.0% to 90.0%, and DeBERTa (B6) from 30.0% to 70.0%. B3 has the lowest ASR at every budget (10.0%, 20.0%, 20.0%, and 40.0% at budgets 50, 100, 200, and 500) but also degrades as the budget grows, so RTM reduces rather than removes adaptive exposure. The Soft-OR rule lets a payload that evades the embedding-based memory still be caught by the detector, and vice versa. These results support H3 descriptively. Memory-aware attackers and MELON (B5) are not reported in E3.

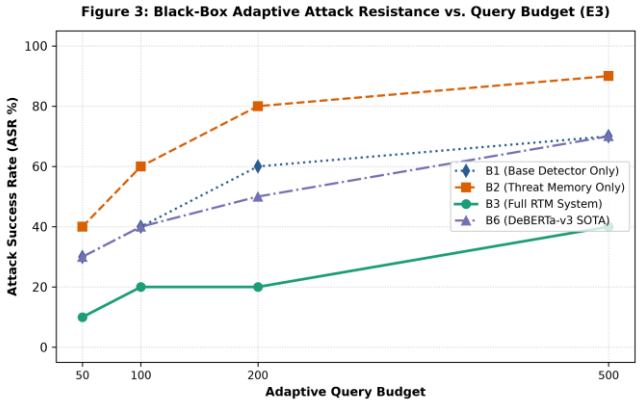


Fig. 3. Attack Success Rate (ASR) vs. Adaptive Query Budget under black-box mutation fuzzing (Experiment E3).

TABLE V
ASR UNDER BLACK-BOX FUZZING BY QUERY BUDGET

Budget	B1	B2	B3	B6
50	30.0%	40.0%	10.0%	30.0%
100	40.0%	60.0%	20.0%	40.0%
200	60.0%	80.0%	20.0%	50.0%
500	70.0%	90.0%	40.0%	70.0%

Each cell: ASR over n = 10 mutated attack variants per system and budget.

D. Memory Poisoning (E4)

Table VI details the poisoning evaluation, which comprises 200 trials in two tracks: Track A, 100 candidate records inserted through the ingestion path (inactive until verified), and Track B, 100 records crafted to mimic benign schema and to pass verification. None of the 100 crafted records passed the automated and analyst verification, so none entered the active memory. Both tracks cover the same five categories (false threat descriptions, contradictory/conflict rules, out-of-scope benign signatures, stale/deprecated patterns, and malicious crafted perturbations) with 20 attempts per category. No attempt changed a decision on the probe set: MPS_cand = 0/100 and MPS_craft = 0/100 (95% Wilson upper bound 3.7% each; 1.9% over all 200 trials). MPS_cand = 0 follows by construction from keeping candidates inactive, and MPS_craft = 0 shows that the verification gate blocked every crafted record. These zeros do not measure how retrieval and fusion behave when a poisoned record has been admitted to the active memory, which remains untested; the second criterion of H6, which concerns records that pass verification, is therefore not tested (Table VIII).

TABLE VI
MEMORY POISONING SUCCESS BY TYPE

Poison Attack Category	Candidate Trials	Crafted Trials	MPS_cand / MPS_craft ↓
False Threat Descriptions	20	20	0/20 | 0/20
Contradictory / Conflict Rules	20	20	0/20 | 0/20
Out-of-Scope Benign Signatures	20	20	0/20 | 0/20
Stale / Deprecated Patterns	20	20	0/20 | 0/20
Malicious Crafted Perturbations	20	20	0/20 | 0/20

E. Explanation Quality and Privacy (E5)

Two reviewers, who were blinded to the configuration, judged 50 rejection explanations, taken sequentially as the first 50 documented blocks of the E1 log. The Explanation Verification Rate (EVR) was 94.0% (47/50; 95% Wilson CI [83.8%, 97.9%]), with Cohen's $\kappa = 1.000$ (observed agreement 50/50). Because 94% of the labels are positive, κ is sensitive to prevalence and is best read together with the raw agreement. The automated privacy audit recovered no protected context from the redacted explanations in 0/50 attempts (95% upper bound 7.1%).

F. Ablations and Latency (E6)

Table VII reports the ablations. They were run on a small evaluation subset of 12 attack and 10 benign items, so each percentage step corresponds to a single item and the table is descriptive. It is not computed on the full partitions of Table III, which explains why the Full RTM row differs from Table III. The non-monotone FPR changes across ablations arise from the small subset (n = 22), in which the isotonic calibration that yields 0.0% FPR for Full RTM may not generalize; these values are therefore descriptive and not extrapolated to the full partition. Relative to Full RTM (ASR = 8.3%, FPR = 0.0%, F1 = 0.957), removing the memory raises ASR to 25.0% and removes the explanations (EVR is undefined); removing the detector raises ASR to 41.7%; removing calibration raises FPR to 30.0%; and removing segmentation or exemplars raises ASR to 25.0%, with EVR falling to 75.0% without exemplars. Replacing the verified

memory with random benign records (B3-rand) gives FPR = 40.0% and F1 = 0.720, and BM25 retrieval (B3-lex) gives ASR = 25.0% and EVR = 0.0%. Across k ∈ {1, 3, 5, 10}, ASR ranges from 8.3% (k = 5) to 25.0% (k = 10).

Latency and two-stage cascade. Table III reports the latency of the local stage of B3 (median 22.0 ms, P95 38.6 ms), measured with cached detector scores (Section V-D); it covers embedding, FAISS retrieval, and fusion, and equals that of B2, which has no detector (21.7 ms). B1 (1,719.9 ms median, P95 25,229.3 ms) and MELON (B5: 1,102.9 ms median, P95 1,246.1 ms) are measured live, so the figures are not like-for-like. Stage 1 resolved 0.0% of the held-out attacks in E1 (maximum $S_{sim} = 0.57 < \tau_{block} = 0.80$), so every attack was passed to the detector. Consequently, the end-to-end latency of B3 on these items is about 22.0 ms plus that of B1 (roughly 1.74 s at the median, about 1.3% above B1 and about 58% above MELON's 1,102.9 ms). At the 95th percentile the gap is far larger, because B1 alone reaches 25,229.3 ms against 1,246.1 ms for MELON. The 22.0 ms is therefore a local retrieval overhead and not a production latency, and Stage 1 can save detector calls only for threats that lie close to documented records.

TABLE VII

ABLATION RESULTS (SUBSET OF 12 ATTACK AND 10 BENIGN ITEMS)

Configuration	ASR ↓	FPR ↓	F1 ↑	EVR ↑
Full RTM (B3)†	8.3%	0.0%	0.957	94.0%
− memory	25.0%	10.0%	0.818	N/A
− detector	41.7%	10.0%	0.700	94.0%
− calibration	16.7%	30.0%	0.800	94.0%
− segmentation	25.0%	10.0%	0.818	94.0%
− exemplars	25.0%	10.0%	0.818	75.0%
k = 1	16.7%	10.0%	0.870	94.0%
k = 3	16.7%	10.0%	0.870	94.0%
k = 10	25.0%	10.0%	0.818	94.0%
B3-rand	25.0%	40.0%	0.720	0.0%
B3-lex	25.0%	10.0%	0.818	0.0%

† Computed on the ablation subset (12 attack and 10 benign items); not comparable with the full-partition values of Table III. Non-monotone FPR shifts across configurations reflect subset sample size rather than asymptotic behavior.

‡ EVR values of 94.0% for configurations retaining the memory reflect the full E5 audit baseline (47/50, 94.0%); within the 22-item ablation, the drop to 75.0% without exemplars and 0.0% for lexical/random controls reflects attribution failure. EVR is not defined for the − memory configuration, because no record is retrieved to cite.

G. Hypothesis Outcomes

Table VIII lists the criteria and the observed outcomes. A hypothesis is marked not supported when its criterion is not met; partially supported when some criteria were met or tested; and supported (point estimate) or supported (descriptive) when the point estimate or the descriptive pattern meets the criterion without a formal test or a confidence bound that does.

TABLE VIII

HYPOTHESES AND OUTCOMES

H	Criterion (Analysis Plan)	Empirical Observation	Verdict
H1	B3 ASR < B1 at equal or lower FPR (paired test, Holm-corrected)	ASR 6.3% (105/1,673) vs. B1 5.7% (96/1,673); FPR 16.1% vs. 16.1%. B1 blocked 9 attacks that B3 permitted; B3 none that B1 permitted (exact McNemar p = 0.0039). Post hoc: B3 within 0.5 pp of B1. B3 better than B0, B2, B6 (p < 0.001).	Not supported
H2	AG > 0 on each held-out family (95% CI lower bound > 0), ΔFPR ≤ 1.0 pp	Pooled AG = +88.9 pp [+72.2, +100.0] (18 cases); per-family lower bound = 0 for financial_data and physical_data; ΔFPR = +0.00 pp	Partially supported
H3	B3 ASR < B1 and B2 under black-box fuzzing at all budgets	B3: 10%, 20%, 20%, 40% vs. lowest baseline 30%, 40%, 50%, 70% (also below B6) at budgets 50/100/200/500; n = 10 per cell, no significance test	Supported (descriptive)
H4	EVR ≥ 90%, κ ≥ 0.60, zero privacy leakage	EVR = 94.0% (47/50); κ = 1.000; 0/50 protected-context leaks (95% upper bound 7.1%)	Supported (point estimate)
H5	TSR within 5.0 pp of B0; non-inferior to B5 (margin Δ = 5.0 pp); lower latency	TSR Δ = 0.0 pp (agent level, ≤ 24 tasks). ASR_B3 − ASR_B5 = −6.2 pp, one-sided 95% upper bound +4.9 pp (p ≈ 0.049; exploratory, B5 n = 24). Latency: local stage 22.0 ms, but about 1.74 s end to end for attacks (Stage 1 resolved 0.0%) vs. 1.10 s for MELON	Not supported

H6	MPS_cand = 0% (by construction); MPS_craft ≤ 5% for crafted records that pass verification	MPS_cand = 0/100 (by construction). 0/100 crafted records passed verification, so MPS_craft = 0/100 (95% upper bound 3.7%) measures the gate; admitted poisoned records were not tested	Partially supported

VII. DISCUSSION

Adaptation versus Memorization. A central question for retrieval-based defenses is whether gains reflect generalization or memorization. Three observations bear on this question, although each rests on a small sample and none is conclusive. First, the E2 adaptation gain is measured on held-out variants disjoint from the ingested exemplars (18 paired cases). Second, in the ablation subset, replacing the verified memory with random benign records (B3-rand) raises FPR to 40.0% (F1 = 0.720), and BM25 retrieval (B3-lex) yields ASR = 25.0% and no verifiable explanations (EVR = 0.0%). Third, updating the base detector with a few-shot example (B4) reached ASR = 0.0% on its pilot sample of 10 attack items, so the memory is not shown to be more accurate than a detector-side update; its advantages are the explanation tied to a documented record and an update that leaves the detector prompt unchanged. Larger E2 and B3-versus-B4 comparisons are needed before claiming that the memory outperforms prompt-based updating.

Comparison with MELON. MELON inspects agent behavior by comparing the tool calls of the original and a masked re-execution, which adds a second inference pass per tool return (median 1,102.9 ms, P95 1,246.1 ms in our runs). RTM operates at the tool-return boundary: its local stage costs 22.0 ms and uses no LLM call, but its Stage 2 detector call costs about as much as B1 (1,719.9 ms median), so RTM is not faster than MELON end to end for items that reach Stage 2. The B3–B5 comparison rests on 24 trials and a different population and is exploratory. MELON returns a binary decision, whereas RTM supplies a citation to a documented record and the triggering segment. RTM is therefore better suited where auditable evidence and updatable knowledge are required, and behavioral re-execution remains a complementary layer for high-consequence actions.

False Positives. The FPR of RTM on benign content (16.1%) equals that of B1 and is far above the 1% budget used for calibration. The FPR is 26.3% (46/175; 95% Wilson CI [20.3%, 33.3%]) on the 175 hard negatives (advisories, instructional text, and code with legitimate imperative sentences) and 3.5% (5/142; [1.5%, 8.0%]) on the 142 other benign items. Hard negatives are scarce in the calibration partition, so calibration under-penalizes them. An FPR of this size would disrupt interactive agents, and reducing it is the main open problem of this work; the 42.9% FPR of the public DeBERTa classifier on the same items indicates that the problem is shared by other detectors, although this does not reduce its impact on interactive agents. Prevalence matters as well: 84% of the test items are attacks, whereas at a realistic attack prevalence of 1% the same recall (93.7%) and FPR (16.1%) would yield a precision of only about 5.6%.

Failure Analysis and Design Implications. First, residual evasions of B3 (105/1,673) tend to involve nested instructions and heavily obfuscated payloads that lie far from the memory exemplars in embedding space and also receive a low detector score. The 105 residual evasions are distributed across the six families as follows: data_security_harm (21), financial_data (18), financial_harm (17), language_translation (16), physical_data (17), and physical_harm (16). Second, in E2, ingesting a single exemplar for a family removed these evasions in all 18 cases, which suggests that residual failures may be remediable by memory updates without fine-tuning, subject to the small sample. Third, removing calibration raised FPR from 0.0% to 30.0% in the ablation subset, which indicates that calibration limits false alarms. Fourth, because candidate records stay inactive until verified and the gate blocked every crafted record, no poisoning attempt changed a decision; this underscores the need for a verification gate and human review of memory updates, whereas the effect of an admitted poisoned record is untested.

VIII. THREATS TO VALIDITY AND LIMITATIONS

Construct validity. Embedding similarity may measure lexical or topical proximity and not malicious behavior, and benign content that discusses prompt injection can cause false positives; such content is included in the benign partition as hard negatives. **Internal validity.** An apparent gain can arise from a changed threshold, a different model, or data leakage; the model, prompts, tools, test cases, and calibration are fixed during the before-and-after experiment, and detector run-to-run variance was not estimated (see Statistical conclusion validity). **External validity.** AgentDojo does not represent every agent, language, modality, or high-impact domain, and results on one closed model do not establish universal robustness. The detector LLM may also have been exposed to the public benchmarks during pretraining.

Statistical conclusion validity. Several experiments rest on small samples: E2 (18 paired cases), E3 (n = 10 per cell), the ablations (12 attack and 10 benign items), B4 (10 attack and 10 benign items), B5 (24 attack and 20 benign trials), and E5 (50 explanations, taken sequentially and judged by the two authors, so that κ may be optimistic). E1, E2, and E6 were run once with deterministic decoding, so run-to-run variance is not estimated. Intervals are wide, differences between table cells are not tested, and these results should be read as indicative. Item-level ASR on static benchmark items differs from agent-level ASR, and the held-out attack variants are generated by the authors' own mutation procedure, which may favor or disfavor particular defenses. Variants within a family share templates and seeds, and the 18 cases of E2 are clustered within six families, so intervals that assume independent items may be too narrow.

Limitations. The scope is an agent that handles textual or retrieved content, with one decision per content item. The memory depends on the quality and timeliness of its sources, and ingestion is semi-automated with verification before insertion by design, not fully live. RTM may miss subtle logical manipulation that contains no explicit instruction, and it may not generalize to multimodal attacks without additional components. The detector is itself an LLM and inherits residual injection risk. RTM is a complementary layer and not a replacement for execution-centric controls such as least privilege, tool isolation, and human approval. Adaptive attackers may also exploit the memory by probing which record was matched; mitigations include uniform

responses to untrusted parties, rate limiting, and reporting ASR as a function of query budget. The false positive rate on benign content (16.1%) limits use in interactive settings. The poisoning evaluation tests the verification gate, not retrieval and fusion with an admitted poisoned record.

IX. ETHICAL CONSIDERATIONS

All attacks are executed in synthetic environments or isolated test accounts, with no access to private user data, external messages, production files, or irreversible actions. Payloads are stored and shared with care and no operational secrets enter the memory or explanations. The objective is to measure defensive behavior, not to compromise real systems. Furthermore, several commercial safeguard classifiers (e.g., Microsoft PromptShield, Meta Prompt Guard, DataSentinel) and structural mitigations (Spotlighting, Instruction Hierarchy) were not benchmarked due to proprietary model access or lack of standardized boundary APIs; comprehensive evaluation against these defenses is planned for future work.

X. CONCLUSION

This paper presented RAG-Based Threat Memory (RTM), a model-centric defense that separates adaptive security knowledge from a training-free base detector through an updatable, verified, retrieval-augmented memory, and ties each rejection to a documented threat and a specific text segment. On 1,673 held-out attack variants and 317 benign items (n = 1,990 per system), RTM reached ASR = 6.3% [95% Wilson CI: 5.2%, 7.5%] at FPR = 16.1% [12.5%, 20.5%], close to the base detector alone (5.7% ASR) and clearly better than a public trained detector (17.3% ASR, 42.9% FPR). Ingesting verified exemplars lowered ASR on six held-out families from 88.9% to 0.0% (18 paired cases) with no FPR increase; none of the 100 crafted poisoning records passed verification and no candidate record changed a decision in 200 trials (95% upper bound 1.9%); and 94.0% of 50 audited explanations were verified. Of the six hypotheses, H4 was supported; H2, H3, and H6 were supported in part or descriptively; and H1 and H5 were not supported, because RTM did not improve on the ASR of the base detector and, since every unseen attack reached the detector, was not faster end to end than MELON. The retrieval stage adds 22.0 ms per item on top of the detector call. The limitations in Section VIII, in particular the 16.1% false positive rate and the small samples behind E2–E5, bound these conclusions.

Future work includes lowering the false positive rate on hard negatives, studying RTM together with execution-centric controls as layers of a defense-in-depth architecture, stricter verification for admitting new threats that reduces manual review without weakening poisoning resistance, evaluation on additional benchmarks, agent types, and languages, and subtle logical-manipulation attacks that contain no explicit instruction.

DATA AND CODE AVAILABILITY

Code, memory records (with non-operational exemplars), partition definitions, configuration files, and per-case logs (cases.csv and cases_e1_expanded.csv) are available at https://github.com/eman232001/RAG_Based-Threat-Memory. Payloads that could be directly harmful are withheld.

ACKNOWLEDGMENT

The author thanks the Faculty of Computer and Information Technology at Sana'a University for supporting this research. In accordance with IEEE policy, the author discloses that generative AI tools were used exclusively for language polishing and formatting verification; all experimental design, code implementation, data auditing, and empirical analyses were conducted and verified by the author.

REFERENCES

[1] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, "Adversarial machine learning: A taxonomy and terminology of attacks and mitigations," National Institute of Standards and Technology, NIST AI 100-2 E2025, 2025. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-2e2025

[2] OWASP GenAI Security Project, "LLM01:2025 Prompt Injection," 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/. Accessed: Oct. 16, 2026.

[3] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in NeurIPS ML Safety Workshop, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[4] Y. Liu et al., "Prompt injection attack against LLM-integrated applications," arXiv preprint arXiv:2306.05499, 2023. [Online]. Available: https://arxiv.org/abs/2306.05499

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[7] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[8] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings of the Association for Computational Linguistics: ACL, 2024. [Online]. Available: https://arxiv.org/abs/2403.02691

[9] J. Yi et al., "Benchmarking and defending against indirect prompt injection attacks on large language models," arXiv preprint arXiv:2312.14197, 2023. [Online]. Available: https://arxiv.org/abs/2312.14197

[10] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[11] Z. Wang et al., "AgentVigil: Automatic black-box red-teaming for indirect prompt injection against LLM agents," in Findings of the Association for Computational Linguistics: EMNLP, 2025. [Online]. Available: https://aclanthology.org/2025.findings-emnlp.1258/

[12] Q. Zhan et al., "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in Findings of the Association for Computational Linguistics: NAACL, 2025. [Online]. Available: https://aclanthology.org/2025.findings-naacl.395/

[13] K. Hines et al., "Defending against indirect prompt injection attacks with spotlighting," arXiv preprint arXiv:2403.14720, 2024. [Online]. Available: https://arxiv.org/abs/2403.14720

[14] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in Proc. USENIX

Security Symposium, 2025. [Online]. Available: https://arxiv.org/abs/2402.06363

[15] E. Wallace et al., "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv preprint arXiv:2404.13208, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208

[16] H. Inan et al., "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint arXiv:2312.06674, 2023. [Online]. Available: https://arxiv.org/abs/2312.06674

[17] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," arXiv preprint arXiv:2501.15145, 2025. [Online]. Available: https://arxiv.org/abs/2501.15145

[18] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in Proc. IEEE Symposium on Security and Privacy (S&P), 2025. [Online]. Available: https://arxiv.org/abs/2504.11358

[19] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in Proc. Int. Conf. Machine Learning (ICML), 2025. [Online]. Available: https://arxiv.org/abs/2502.05174

[20] T. Zhang et al., "AgentSentry: Mitigating indirect prompt injection in LLM agents via temporal causal diagnostics and context purification," arXiv preprint arXiv:2602.22724, 2026. [Online]. Available: https://arxiv.org/abs/2602.22724

[21] E. Debenedetti et al., "Defeating prompt injections by design," arXiv preprint arXiv:2503.18813, 2025. [Online]. Available: https://arxiv.org/abs/2503.18813

[22] T. Shi et al., "Progent: Programmable privilege control for LLM agents," arXiv preprint arXiv:2504.11703, 2025. [Online]. Available: https://arxiv.org/abs/2504.11703

[23] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in Proc. Annual Meeting of the Association for Computational Linguistics (ACL), 2025. [Online]. Available: https://arxiv.org/abs/2412.16682

[24] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[25] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in Proc. USENIX Security Symposium, 2025. [Online]. Available: https://arxiv.org/abs/2402.07867

[26] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in Advances in Neural Information Processing Systems (NeurIPS), 2024. [Online]. Available: https://arxiv.org/abs/2407.12784

[27] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. USENIX Security Symposium, 2024. [Online]. Available: https://arxiv.org/abs/2310.12815

[28] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv preprint arXiv:2307.15043, 2023. [Online]. Available: https://arxiv.org/abs/2307.15043

[29] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[30] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[31] MITRE, "ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems." [Online]. Available: https://atlas.mitre.org. Accessed: Oct. 16, 2026.

[32] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2017. [Online]. Available: https://arxiv.org/abs/1706.04599

[33] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[34] L. Wang et al., "Multilingual E5 text embeddings: A technical report," arXiv preprint arXiv:2402.05672, 2024. [Online]. Available: https://arxiv.org/abs/2402.05672

[35] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[36] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[37] ProtectAI, "deberta-v3-base-prompt-injection-v2," Hugging Face model card. [Online]. Available: https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2. Accessed: Oct. 16, 2026.

[38] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960.

[39] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209–212, 1927.

[40] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[41] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.